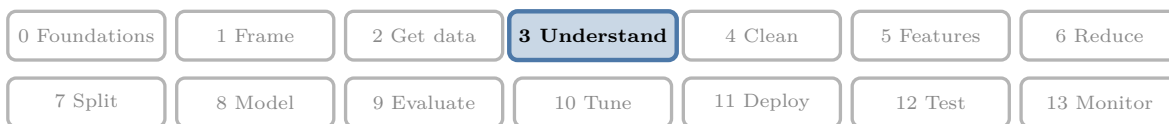


Density Estimation: Parametric and Kernel Density Estimation

Contents

1. Overview	2
2. What density estimation is	3
3. Parametric density estimation	4
3.1 The steps	4
3.2 An example	4
3.3 Why it is called parametric	5
4. Non-parametric density estimation	6
5. Kernel density estimation (KDE)	6
5.1 How a KDE is built	7
5.2 The bandwidth	8
5.3 KDE in scikit-learn	10
5.4 KDE in seaborn	11
6. PDF, histogram and the population	11
7. Summary	13
8. Sources	13
9. Key terms	13

Where this fits: Pipeline map step 3 (Understand data). See the [Course map](#).



- **Builds on:** Population, sample, parameter and statistic ([Note 220](#)); Probability density function (PDF) ([Note 242](#)).
- **Compare with:** Gaussian mixture model (GMM) ([Note 640](#)).

1. Overview

Key point: Density estimation builds a PDF from data: either assume a famous distribution and estimate its parameters, or build the curve from every data point with a kernel density estimate.

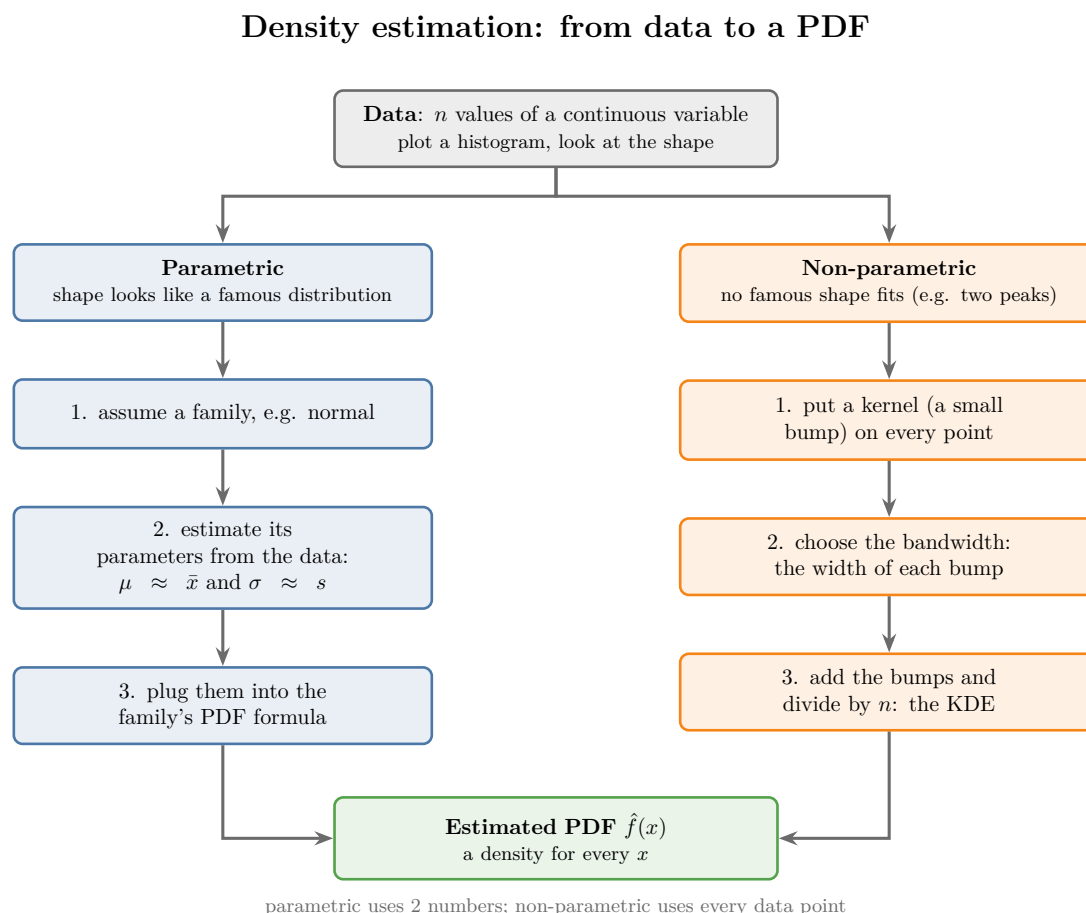


Figure 1: The two routes from data to an estimated PDF

For a discrete variable, the PMF comes straight from counting (see the [PMF and discrete CDF Note](#)). For a continuous variable, the y axis of the PDF is a density, not a probability

(see the [PDF and continuous CDF Note](#)), and densities cannot be counted. They have to be estimated.

Figure 1 shows the two routes. The left one assumes the data follows a famous distribution; the right one makes no assumption. This Note follows both, with code.

2. What density estimation is

Key point: Density estimation is the family of techniques that estimate the PDF of a random variable from a set of observations.

Density estimation is a statistical technique for estimating the probability density function of a random variable from a set of observations (data). An **observation** is one record, one row of the data table; a **feature** is one variable, one column of that table. In simpler terms, it estimates the **underlying distribution**: the distribution that produced the data points.

Density estimation is used in:

- **Data analysis and visualisation:** plotting the PDF shows the shape of the data.
- **Hypothesis testing:** some tests compare data against a distribution, such as the Kolmogorov-Smirnov test (SciPy `kstest` docs).
- **Machine learning:** to estimate the distribution of the input data, or how likely certain events or outcomes are. Gaussian Naive Bayes, for example, estimates one normal density per class and feature (see the [Gaussian Naive Bayes Note](#)).

The methods come in two families, the two routes of Figure 1:

- **Parametric methods** assume the data follows a specific distribution, such as the normal distribution, and estimate its parameters.
- **Non-parametric methods** make no such assumption and estimate the density directly from the data.

Common techniques include kernel density estimation (Section 5), histogram-based estimation, and the Gaussian mixture model. The choice depends on the data and on what the estimate is for.

Extra: A **Gaussian mixture model** assumes the density is a weighted sum of a few normal curves (say two, for data with two peaks) and estimates each curve's mean, standard deviation and weight. A mixture model has a fixed set of parameters, like a parametric method, yet it can describe data with several peaks, which one normal curve cannot. Gaussian mixture models are also closely related to k-means clustering (MML §11.1, §11.5).

3. Parametric density estimation

Key point: Look at the histogram, pick the famous distribution it resembles, estimate that distribution's parameters from the sample, and plug them into its PDF formula.

Parametric density estimation estimates the PDF by assuming that the data comes from a specific parametric family of distributions (normal, exponential, and so on). Once the family is chosen, only its parameters are unknown, and the data is used to estimate them.

3.1 The steps

Key point: Histogram, then assumption, then parameters, then the PDF formula evaluated at many points.

Take the CGPAs of 1,000 students, or any 1,000 measurements of a continuous variable. To build their PDF:

1. **Look at the shape.** Plot a histogram. If it looks like a bell curve, assume the data is normal.
2. **Estimate the parameters.** The normal distribution has two, μ and σ . We estimate them with the sample mean $\bar{x}$ and the sample standard deviation s (statistics standing in for parameters, see the [what is statistics Note](#), section 4.2).
3. **Plug them into the PDF formula.** With μ and σ fixed, the normal PDF has only x left unknown. Feeding it any x returns a density.
4. **Draw the curve.** Evaluate the PDF at many points, say 100 evenly spaced values from the smallest to the largest data value, and join them.

3.2 An example

Key point: 1,000 values from a normal distribution with mean 50 and standard deviation 5 give estimates of 49.86 and 4.94; the fitted curve follows the histogram.

We generate 1,000 values from a normal distribution with $\mu = 50$ and $\sigma = 5$, then pretend we do not know where they came from. The values run from 31.76 to 65.89, and the histogram looks like a bell, so we assume a normal distribution.

1. **In words:** estimate μ and σ by the sample's mean and standard deviation, then compute the normal PDF with those values.
2. **Formula:**

$$\hat{f}(x) = \frac{1}{s\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\bar{x}}{s}\right)^2}$$

3. **Example:** the sample gives $\bar{x} = 49.86$ and $s = 4.94$, close to the true 50 and 5 but not equal, because a sample is not the population. At $x = 50$:

$$\hat{f}(50) = \frac{1}{4.94\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{50-49.86}{4.94}\right)^2} = 0.0807$$

The true density at 50 is 0.0798.

Figure 2 shows the fitted curve (orange) over the histogram. The fitted curve follows the bars closely.

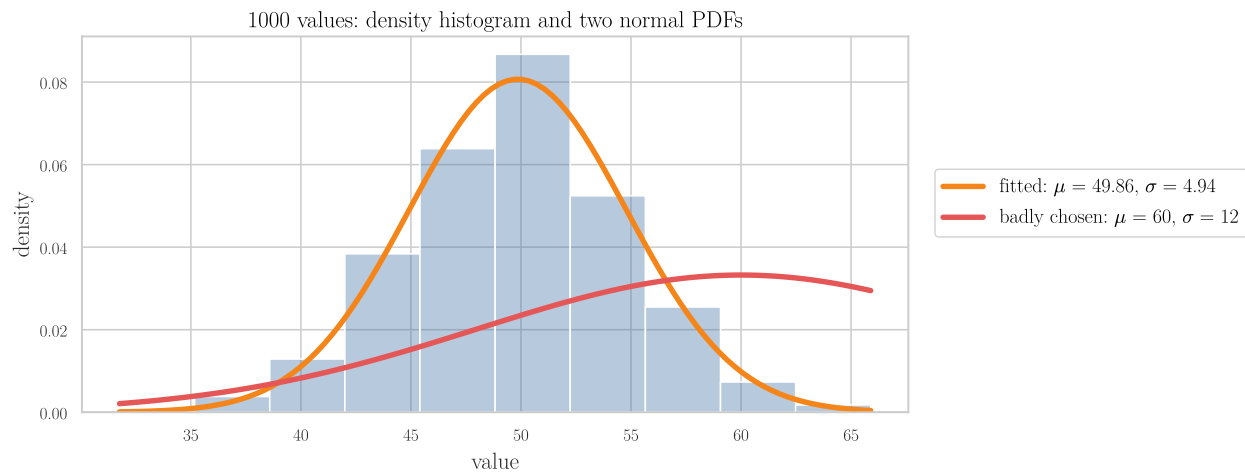


Figure 2: 1,000 values: the density histogram, the fitted normal PDF and a normal PDF with badly chosen parameters

The histogram must be drawn on the **density** scale (bar area = share of the data, as in Figure 1 of the [PDF and continuous CDF Note](#)). A histogram of counts would have bars about 1,000 times taller than the curve, and the two could not be compared.

Python: Fitting a normal distribution.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(42)
sample = rng.normal(loc=50, scale=5, size=1000)
# estimate the two parameters
mu, sigma = sample.mean(), sample.std()
dist = stats.norm(mu, sigma)
# 100 points from min to max, one density each
values = np.linspace(sample.min(), sample.max(), 100)
densities = dist.pdf(values)
dist.pdf takes a whole array at once, so no loop is needed. To compare with the histogram
in Plotly, use histnorm="probability density"; in seaborn, stat="density". The
Notebook (notebook.ipynb) draws both.
```

3.3 Why it is called parametric

Key point: The whole curve depends on two estimated numbers; bad estimates give a bad curve, and more data gives better estimates.

The curve depends only on the two parameters. Change them and the curve changes completely. Figure 2's red curve uses $\mu = 60$ and $\sigma = 12$, far from the data: its peak is in the wrong place and it is much too flat. At $x = 50$ it gives 0.0235 instead of about 0.08.

So the whole game is estimating the parameters well. The more data we have, the closer the sample's mean and standard deviation get to the population's, and the better the PDF. With little data, the estimates, and so the curve, can be poor.

Extra: `sample.std()` in NumPy divides by n , not $n - 1$. With 1,000 values the difference is tiny (4.944 against 4.946); see Bessel's correction in the [measures of dispersion Note](#).

Extra: Older code fits this kind of curve with `sns.distplot`, which seaborn has deprecated (seaborn 0.13 docs). `distplot` drew a KDE (Section 5) by default (`kde=True`, `fit=None`), not a fitted normal, so its curve can differ slightly from the parametric one even on the same data. Today we use `sns.histplot(x, stat="density", kde=True)` for a histogram with a KDE, and compute a fitted normal with `scipy.stats.norm` as above.

4. Non-parametric density estimation

Key point: When no famous distribution fits, non-parametric methods estimate the PDF from the data points themselves, with no assumption about the shape.

Parametric estimation needs the data to resemble a famous distribution. Sometimes it does not: the histogram is not normal, not uniform, not log-normal, not anything with a name. Data with two peaks is a common example. Then the assumption of step 1 fails, and the method cannot be used.

Non-parametric density estimation estimates the PDF of a random variable without assuming any underlying distribution. The method needs no predefined distribution function. Instead of summarising the data by a few parameters (a mean and a standard deviation), it uses **every data point** to build the curve.

- **Advantage:** no assumption about the shape, so it works for any data.
- **Disadvantages:** it is **computationally intensive**, since every point takes part in every density value, and it needs **more data** to give an accurate estimate.

The most common non-parametric method is the kernel density estimate.

5. Kernel density estimation (KDE)

Key point: KDE places a small bump (a kernel) on every data point and adds the bumps up; the sum, divided by the number of points, is the estimated PDF.

The kernel density estimate (KDE) is the smooth curve drawn over a histogram in the [univariate analysis Note](#) (section 7). **Kernel density estimation** is the technique behind it: it uses a kernel function to smooth out the data into a continuous estimate of the density.

5.1 How a KDE is built

Key point: One Gaussian bump centred on each point, all of the same width; at every x , add the bumps' heights.

Take six data points: 2, 2.5, 3, 4, 8 and 8.5. Their histogram (Figure 3, left) shows two groups with nothing in between. No famous distribution has that shape, so we use a KDE.

1. **Choose a kernel.** A **kernel** is a small, symmetric bump with area 1. The most used one is the **Gaussian kernel**: the normal curve.
2. **Put one kernel on every point.** Each data point becomes the centre (mean) of its own normal curve (Figure 3, right, dotted).
3. **Give all kernels the same width.** The standard deviation of each bump is the **bandwidth**, h . The bandwidth is a setting we choose.
4. **Add up.** At every x , add the heights of all the bumps at that x . Where many points sit close together, many bumps overlap and the sum is high. Dividing by the number of points keeps the total area at 1.

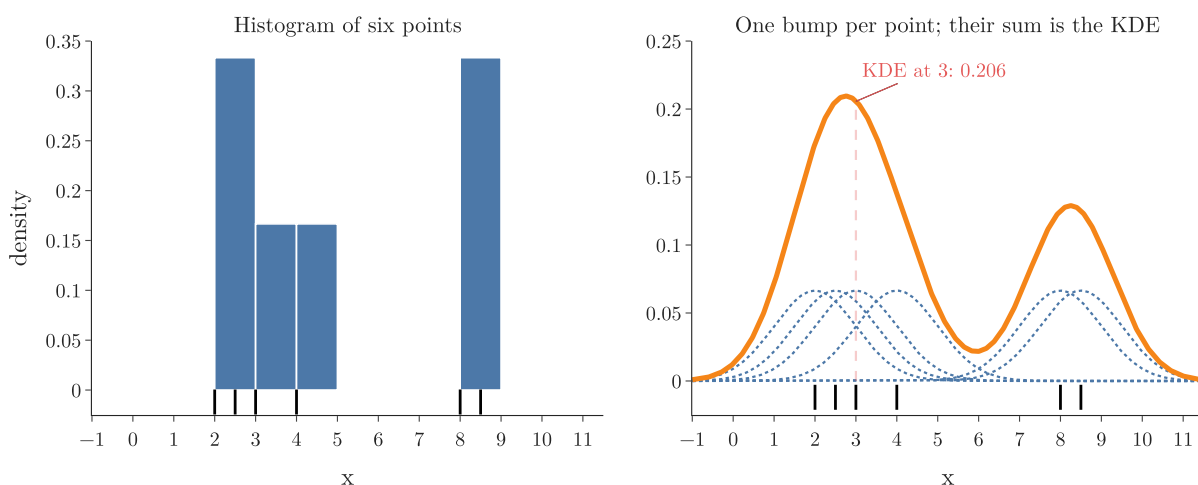


Figure 3: KDE by hand: a Gaussian bump of bandwidth 1 on each of six points (dotted); their scaled sum is the KDE (orange)

1. **In words:** the KDE at x is the average, over all data points, of a normal curve centred on that point and evaluated at x .
2. **Formula:** for n points $x_1, \dots, x_n$ and bandwidth h , with the standard normal curve

$$\phi(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$$

the KDE is

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n \phi\left(\frac{x - x_i}{h}\right)$$

3. **Example:** at $x = 3$ with $h = 1$, the six bumps have heights

$$\phi(1) = 0.242, \phi(0.5) = 0.352, \phi(0) = 0.399, \phi(-1) = 0.242, \phi(-5) \approx 0, \phi(-5.5) \approx 0$$

Their sum is 1.235, so

$$\hat{f}(3) = \frac{1.235}{6 \times 1} = 0.206$$

The points 8 and 8.5 are too far away to contribute.

The result has two peaks, matching the two groups. No formula was assumed anywhere: the shape came from the points.

Extra: Why divide by nh ? Each bump

$$\frac{1}{h} \phi\left(\frac{x - x_i}{h}\right)$$

is a normal PDF with standard deviation h , so it has area 1. Adding n of them gives area n ; dividing by n brings the total back to 1, as every PDF needs.

5.2 The bandwidth

Key point: A small bandwidth gives thin bumps and a spiky curve; a large one gives wide bumps and a smooth curve that can blur real features. The right value is found by trying.

Figure 4 first builds the KDE of the six points one bump at a time, then turns the bandwidth. Watch the orange sum: with thin bumps it becomes six separate spikes, and with wide bumps the two groups melt into one hill.

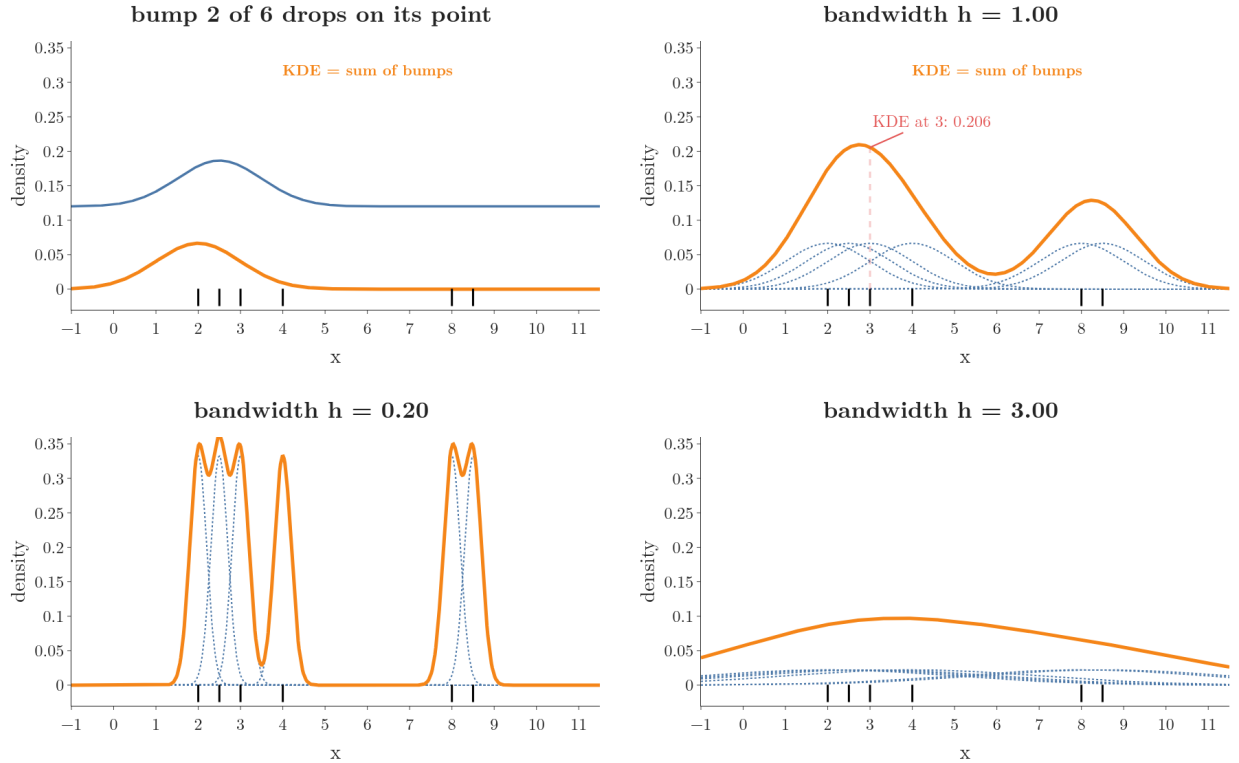


Figure 4: The KDE of section 5.1 built step by step: a bump of bandwidth 1 drops onto each of the six points and the orange sum grows (0.206 at $x = 3$). Then the bandwidth sweeps from 0.2 (six spikes) to 3 (one hill)

The bandwidth sets how wide each bump is, and so how smooth the KDE is:

- **Small bandwidth:** thin, tall bumps. Each point shows up as its own spike, and the curve is irregular. The curve follows the noise of this particular sample.
- **Large bandwidth:** wide, low bumps that overlap a lot. The curve is smooth, but too much smoothing hides real features.

Figure 5 shows this on 1,000 values with two peaks: 300 values around 20 and 700 around 40.

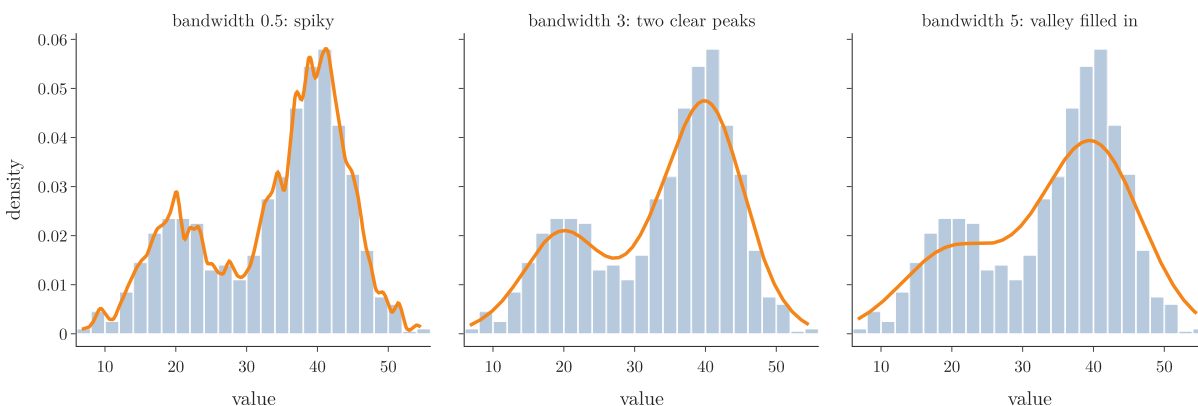


Figure 5: KDE of two-peaked data with bandwidth 0.5, 3 and 5

- **Bandwidth 0.5:** a jagged curve with many small peaks that are only noise.
- **Bandwidth 3:** two clear peaks and a valley between them.
- **Bandwidth 5:** smooth, but the valley is filled in and the left peak turns into a shoulder. The density at 30 rises from 0.0126 (bandwidth 0.5) to 0.0224, now higher than the density at 20 (0.018).

There is no single correct bandwidth. We try a few values and keep the one that shows the shape without the noise, as with the number of bins of a histogram.

Extra: The Gaussian is not the only kernel. scikit-learn also offers "tophat" (a flat box), "epanechnikov" (a downward parabola), "exponential", "linear" (a triangle) and "cosine". The choice of kernel matters much less than the bandwidth: the common kernels are all close in quality, the Gaussian reaching about 95% of the best one, Epanechnikov (Silverman 1986, Table 3.1). The Gaussian is scikit-learn's default.

5.3 KDE in scikit-learn

Key point: `KernelDensity(bandwidth=...)` is fitted on a 2D array; `score_samples` returns the log of the density, so we apply `np.exp`.

Python: KDE with scikit-learn.

```
from sklearn.neighbors import KernelDensity

model = KernelDensity(kernel="gaussian", bandwidth=3)
# scikit-learn wants a 2D array: one row per value
model.fit(data.reshape(-1, 1))
grid = np.linspace(data.min(), data.max(), 100)
grid = grid.reshape(-1, 1)
# score_samples gives log densities; exp undoes the log
densities = np.exp(model.score_samples(grid))
```

Three details matter:

- **Two settings:** the `kernel` (default "gaussian") and the `bandwidth` (default 1.0).
- **2D input:** scikit-learn models expect a table, one row per sample and one column per feature, so a single column must be reshaped with `reshape(-1, 1)`.
- **Log densities:** `score_samples` returns the **logarithm** of the density, not the density. Far from the data a density can be too small for the computer to store, while its log stays an ordinary number: for the six points, the log density at 60 is -1328.8 , but $e^{-1328.8}$ comes back as exactly 0 (the Notebook shows it). Applying `np.exp`, which undoes a logarithm, gives the densities. For the six points, the KDE at 3 comes back as $\log(0.206) = -1.58$, and $e^{-1.58} = 0.206$.

5.4 KDE in seaborn

Key point: seaborn's `bw_adjust` multiplies a bandwidth seaborn chooses itself; it is not the bandwidth.

seaborn draws a KDE with `sns.kdeplot(x=data)`, or `so.KDE()` in the objects interface. Its smoothness setting is `bw_adjust`, with default 1.

`bw_adjust` is a **multiplier**, not a bandwidth. seaborn first picks a bandwidth with a rule of thumb (Scott's rule, based on the standard deviation and the number of points), then multiplies it by `bw_adjust` (seaborn `kdeplot` docs). For the two-peaked data the rule gives a bandwidth of 2.58. So:

- `bw_adjust=1` means a bandwidth of 2.58, close to scikit-learn's 3.
- `bw_adjust=2` means 5.17; `bw_adjust=5` means 12.9, far smoother than scikit-learn's `bandwidth=5`.
- `bw_adjust=0.02` means 0.05: an extremely spiky curve.

The multiplier explains why the same number gives different curves in the two libraries.

Extra: Scott's rule sets the bandwidth to $h = s \times n^{-1/5}$. For the two-peaked data, $s = 10.29$ and $n = 1000$:

$$h = 10.29 \times 1000^{-1/5} = 10.29 \times 0.251 = 2.58$$

(SciPy `gaussian_kde` docs.) More data gives a smaller bandwidth: with many points, neighbouring thin bumps overlap and fill the gaps, so the curve is no longer spiky. The Notebook checks this with the bandwidth fixed at 0.5: the KDE has about 16 local peaks with 1,000 points but only 3 with 100,000 (the true density has 2).

6. PDF, histogram and the population

Key point: A histogram shows only the sample and depends on the bins; a density estimate aims at the population's shape, but it is only as good as its assumption, its settings and its data.

Why estimate a PDF when a histogram already shows the data? A histogram describes the **sample** we happen to have, and its picture changes with the number of bins. The goal

of statistics is to learn about the **population**. A density estimate tries to describe the population's distribution, from which the sample came.

A good estimate needs the right ingredients:

- **Parametric:** a correct assumption about the family, and enough data to estimate the parameters well. The bin count of the histogram plays no part in the fitted curve.
- **KDE:** a sensible bandwidth, which plays the role the bins play in a histogram, and enough data.
- **Both:** outliers affect the estimate. In a parametric fit an outlier pulls the mean and inflates the standard deviation; in a KDE it adds a small bump of its own.

If the sample does not represent the population, for example because of sampling bias (see the [challenges in ML Note](#)), no density estimate can repair that.

7. Summary

	Parametric	Non-parametric (KDE)
Assumption	data follows a named family	none
Uses	a few parameters ($\bar{x}$, s)	every data point
Setting to choose	the family	kernel and bandwidth
Works on two-peaked data	no (with one normal)	yes
Cost	cheap	computationally intensive
Example	$\hat{f}(50) = 0.0807$	$\hat{f}(3) = 0.206$

- Density estimation turns data into an estimated PDF $\hat{f}(x)$.
- Parametric: histogram, assume a family, estimate its parameters, plug them into the PDF formula.
- KDE: a kernel on every point, bandwidth = kernel width, add and divide by n .
- Small bandwidth: spiky; large bandwidth: smooth, may hide peaks.
- scikit-learn: `score_samples` returns log densities. seaborn: `bw_adjust` multiplies a default bandwidth.

8. Sources

Built from

- CampusX, “Session 40 - Probability Distribution Functions - PDF, PMF & CDF | DSMP 2023”, YouTube, https://www.youtube.com/watch?v=C_QAURbgBqY

Other references

- Deisenroth, M. P., Faisal, A. A. and Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. Sections 11.1 and 11.5 (Gaussian mixture models).
- Silverman, B. W. (1986). *Density Estimation for Statistics and Data Analysis*. Chapman and Hall. Section 3.3.2, Table 3.1 (kernel efficiencies).
- seaborn documentation, `distplot` (deprecated in 0.13) and `kdeplot` (`bw_method`, `bw_adjust`).
- SciPy documentation, `scipy.stats.kstest` and `scipy.stats.gaussian_kde` (Scott’s rule).

9. Key terms

Term	Meaning
Observation	One record, one row of the data table
Feature	One variable, one column of the data table
Density estimation	Estimating the PDF of a random variable from observed data

Term	Meaning
Underlying distribution	The distribution that produced the data points
Parametric density estimation	Assuming a named distribution and estimating its parameters from the data
Non-parametric density estimation	Estimating a PDF from the data with no assumption about its shape
Kernel density estimation	Building a PDF by adding a kernel centred on every data point
Kernel	A small symmetric bump with area 1, placed on each point in a KDE
Gaussian kernel	A kernel shaped like the normal curve; the usual default
Bandwidth	The width (for a Gaussian kernel, the standard deviation) of each kernel; sets the KDE's smoothness
Gaussian mixture model	A density built as a weighted sum of a few normal curves
<code>KernelDensity</code>	scikit-learn's KDE; <code>score_samples</code> returns log densities
<code>bw_adjust</code>	seaborn's multiplier on its default KDE bandwidth
Scott's rule	A rule-of-thumb bandwidth: $s \times n^{-1/5}$